INFORME PROYECTO
ISIS1511 – Fundamentos de Bases de Datos · Entrega 2 · Octubre 2026
Grupo 4: Catalina Saavedra, Esteban Serna, Samuel Fonseca

Punto 1 – Análisis y revisión del modelo de datos

Modificaciones respecto a la Entrega 1

Se creó la entidad WHEELS_USUARIO_ROL, que asocia cada cuenta de autenticación de Oracle APEX con un estudiante registrado y con su rol en la plataforma (PASAJERO, CONDUCTOR o AMBOS). Esta entidad permite identificar al usuario autenticado dentro de los procesos de la aplicación y restringir el acceso a las páginas según su rol. Las demás entidades del modelo se mantienen.
Los modelos E/R y relacional actualizados se anexan en la carpeta docs del repositorio (diagrama E/R y plantilla Excel del modelo relacional).

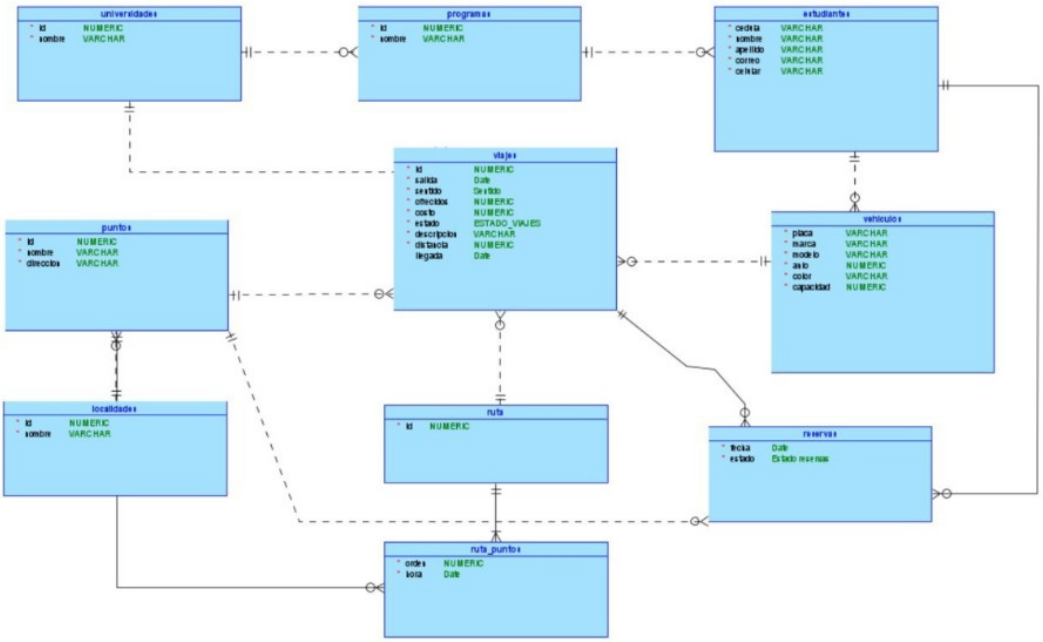


Figura. Modelo E/R de las entidades de negocio de Wheels Alpes. La entidad WHEELS_USUARIO_ROL se relaciona con ESTUDIANTES (1 a 1).

Verificación de la Forma Normal de Boyce-Codd (BCNF)

Una relación está en BCNF si, para toda dependencia funcional no trivial X → Y, X es superclave. A continuación se verifica cada relación del modelo:

Relación	Dependencias funcionales	Claves candidatas	BCNF
UNIVERSIDADES(id, nombre)	id → nombre; nombre → id	{id}, {nombre}	Sí
PROGRAMAS(id, nombre, universidad)	id → nombre, universidad	{id}	Sí
ESTUDIANTES(cedula, nombre, apellido, correo, celular, programa)	cedula → resto; correo → resto	{cedula}, {correo}	Sí
VEHICULOS(placa, marca, modelo, color, estudiante, anio, capacidad)	placa → resto	{placa}	Sí

LOCALIDADES(id, nombre)	id → nombre; nombre → id	{id}, {nombre}	Sí
PUNTOS(id, nombre, direccion, localidad)	id → resto; direccion → resto	{id}, {direccion}	Sí
RUTA(id)	Solo dependencias triviales	{id}	Sí
RUTA_PUNTOS(ruta, punto, orden, hora)	(ruta, punto) → orden, hora; (ruta, orden) → punto, hora	{ruta, punto}, {ruta, orden}	Sí
VIAJES(id, salida, sentido, ofrecidos, costo, estado, descripcion, distancia, llegada, vehiculo, universidad, ruta, punto)	id → resto	{id}	Sí
RESERVAS(estudiante, viaje, punto, fecha, estado_reservas)	(estudiante, viaje) → resto	{estudiante, viaje}	Sí
WHEELS_USUARIO_ROL(apex_username, estudiante_cedula, rol)	apex_username → resto; estudiante_cedula → resto	{apex_username}, {estudiante_cedula}	Sí

En todas las relaciones, el determinante de cada dependencia funcional es una clave candidata, por lo que el modelo cumple BCNF y no fue necesario descomponer ninguna relación. La universidad del estudiante no se almacena en ESTUDIANTES, sino que se obtiene a través de PROGRAMAS; esto evita la dependencia transitiva cedula → programa → universidad y la redundancia que generaría.

Punto 2 – Implementación del esquema de la base de datos

El esquema se implementó en Oracle APEX mediante SQL Workshop, ejecutando el script 01_creacion_tablas.sql. Las restricciones se definieron con nombres explícitos según su tipo: pk_ para claves primarias, fk_ para claves foráneas, ck_ para restricciones de chequeo y unique_/uq_ para unicidad.

Las restricciones de chequeo incluyen los estados válidos de viajes (programado, en curso, completado, cancelado) y de reservas (pendiente, aceptada, rechazada, cancelada), el sentido del viaje, los roles válidos de usuario y los valores positivos de cupos ofrecidos, capacidad, distancia, año y orden de los puntos de la ruta.

Punto 3 – Población de tablas

Las tablas se poblaron con el script 02_poblacion.sql, que genera:

- 5 universidades, 10 programas y 10 localidades de Bogotá.
- 170 estudiantes: los estudiantes 1 a 20 son conductores (un vehículo cada uno) y los estudiantes 21 a 170 son los 150 pasajeros.
- 30 puntos de recogida y 20 rutas de 3 puntos cada una.
- 80 viajes completados (4 por conductor) y 300 reservas aceptadas, asignadas de forma que cada conductor transporta 15 pasajeros distintos.
- Viajes programados, en curso y cancelados, reservas en todos los estados y las cuentas de usuario de prueba (estudiante1 como pasajero y conductor1 como conductor) para verificar los requerimientos funcionales.

El cumplimiento de los mínimos exigidos se verifica con las siguientes consultas:

```
-- Conductores (estudiantes con vehículo): 20
SELECT COUNT(DISTINCT estudiante) FROM vehiculos;

-- Pasajeros (estudiantes sin vehículo): 150
SELECT COUNT(*) FROM estudiantes e
WHERE NOT EXISTS (SELECT 1 FROM vehiculos v WHERE v.estudiante = e.cedula);
```

```
-- Media de pasajeros distintos por conductor: 15
SELECT AVG(n) FROM (
  SELECT vh.estudiante, COUNT(DISTINCT r.estudiante) n
  FROM reservas r
  JOIN viajes v ON r.viaje = v.id
  JOIN vehiculos vh ON v.vehiculo = vh.placa
  GROUP BY vh.estudiante);
```

Punto 4

Nota: el documento marco menciona los requerimientos RFC1 a RFC5, pero solo define RFC1, RFC2 y RFC3; por tanto, se implementaron estos tres. Las consultas se entregan en el archivo 03_consultas_rfc.sql y en la aplicación se presentan de forma visual con filtros por período.

RFC1:

Objetivo: Consultar la cantidad de solicitudes de cupo realizadas en cada localidad durante un período determinado.

Tablas y atributos utilizados:

TABLA	**ATRIBUTOS**
RESERVAS	Punto, fecha
PUNTOS	id, localidad
LOCALIDADES	id, nombre

Justificación de los JOIN:

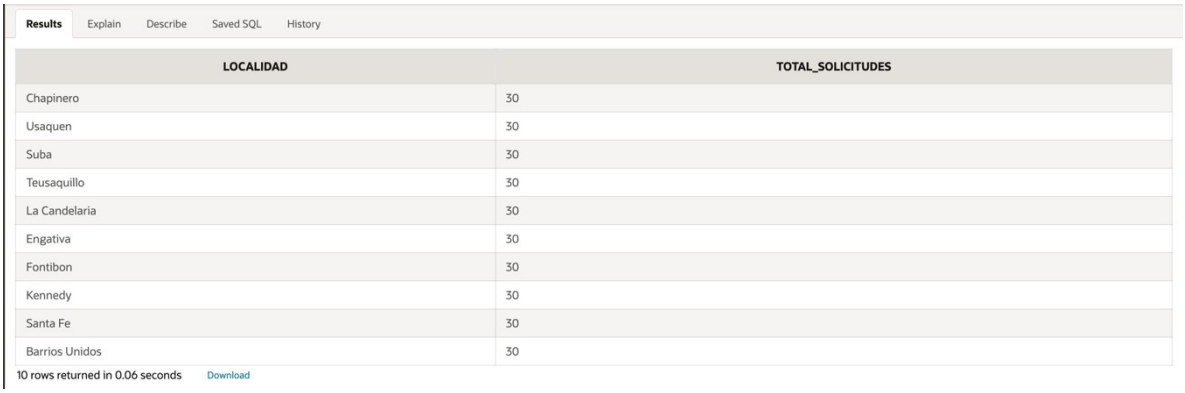
Se utilizaron dos INNER JOIN para relacionar las reservas con los puntos de recogida y, después, con las localidades. Este tipo de unión permite obtener únicamente las solicitudes que tienen un punto de recogida y una localidad asociados. Adicionalmente, se utilizó COUNT(*) para calcular la cantidad de solicitudes y GROUP BY para agruparlas por localidad.

Resultado: La consulta devolvió 300 solicitudes, distribuidas uniformemente entre 10 localidades, con 30 solicitudes por cada una.

Código:

```
SELECT l.nombre AS localidad,
     COUNT(*) AS total_solicitudes
FROM reservas r
INNER JOIN puntos p
   ON r.punto = p.id
INNER JOIN localidades l
   ON p.localidad = l.id
WHERE r.fecha >= DATE '2026-08-25'
  AND r.fecha < DATE '2026-08-26'
GROUP BY l.nombre
ORDER BY total_solicitudes DESC;
```

Resultado:



Results | Explain | Describe | Saved SQL | History

LOCALIDAD	TOTAL_SOLICITUDES
Chapinero	30
Usaquen	30
Suba	30
Teusaquillo	30
La Candelaria	30
Engativa	30
Fontibon	30
Kennedy	30
Santa Fe	30
Barrios Unidos	30

10 rows returned in 0.06 seconds Download

RFC2:
Objetivo: Calcular los ingresos estimados correspondientes a los viajes completados, agrupados por localidad durante un período específico.

Tablas y atributos utilizados:

TABLA	**ATRIBUTOS**
RESERVAS	viaje, punto, estado_reservas
VIAJES	id, costo, salida, estado
PUNTOS	id, localidad
LOCALIDADES	id, nombre

Justificación de los JOIN:

Se utilizaron tres INNER JOIN para relacionar las reservas con sus respectivos viajes, puntos de recogida y localidades. Este tipo de unión permite calcular los ingresos asociados únicamente con reservas que tienen relaciones válidas en las tablas involucradas. Se utilizó SUM(v.costo) para calcular los ingresos estimados, COUNT(*) para contabilizar los pasajeros confirmados y GROUP BY para agrupar los resultados por localidad. Adicionalmente, se filtraron las reservas aceptadas y los viajes completados.

Resultado: Se obtuvieron ingresos estimados totales de $2.400.000, correspondientes a 300 reservas aceptadas, distribuidos en $240.000 por localidad.

Código:

```
SELECT l.nombre AS localidad,
      COUNT(*) AS pasajeros_confirmados,
      SUM(v.costo) AS ingresos_estimados
FROM reservas r
INNER JOIN viajes v
   ON r.viaje = v.id
INNER JOIN puntos p
   ON r.punto = p.id
INNER JOIN localidades l
   ON p.localidad = l.id
WHERE r.estado_reservas = 'aceptada'
```

```
  AND v.estado = 'completado'
  AND v.salida >= DATE '2026-09-01'
  AND v.salida < DATE '2026-09-05'
GROUP BY l.nombre
ORDER BY ingresos_estimados DESC;
```

Resultado:



Results | Explain | Describe | Saved SQL | History

LOCALIDAD	PASAJEROS_CONFIRMADOS	INGRESOS_ESTIMADOS
Chapinero	30	240000
Usaquen	30	240000
Suba	30	240000
Teusaquillo	30	240000
La Candelaria	30	240000
Engativa	30	240000
Fontibon	30	240000
Kennedy	30	240000
Santa Fe	30	240000
Barrios Unidos	30	240000

10 rows returned in 0.10 seconds Download

RFC3:
Objetivo: Consultar la evolución de los viajes completados por cada conductor durante un período determinado, identificando la cantidad de viajes por fecha y localidad.
Tablas y atributos utilizados:

TABLA	**ATRIBUTOS**
VIAJES	id, vehiculo, punto, salida, estado
VEHICULOS	placa, estudiante
ESTUDIANTES	cedula, nombre, apellido
PUNTOS	id, localidad
LOCALIDADES	id, nombre

Justificación de los JOIN:
Se utilizaron cuatro INNER JOIN para relacionar los viajes con los vehículos, los conductores y las localidades asociadas al punto registrado en cada viaje. Esto permite identificar qué conductor realizó cada viaje y en qué localidad se encuentra dicho punto. Se utilizó COUNT(v.id) para contabilizar los viajes y TRUNC(v.salida) para agruparlos por fecha, eliminando la parte correspondiente a la hora.
Resultado: Se identificaron 80 viajes completados correspondientes a 20 conductores, con cuatro viajes realizados por cada conductor entre el 1 y el 4 de septiembre de 2026.

Codigo:

```
SELECT e.cedula AS id_conductor,
    e.nombre || ' ' || e.apellido AS conductor,
    l.nombre AS localidad,
```

```
       TRUNC(v.salida) AS fecha_viaje,
       COUNT(v.id) AS total_viajes
FROM viajes v
INNER JOIN vehiculos vh
   ON v.vehiculo = vh.placa
INNER JOIN estudiantes e
   ON vh.estudiante = e.cedula
INNER JOIN puntos p
   ON v.punto = p.id
INNER JOIN localidades l
   ON p.localidad = l.id
WHERE v.estado = 'completado'
  AND v.salida >= DATE '2026-09-01'
  AND v.salida < DATE '2026-09-05'
GROUP BY e.cedula,
         e.nombre,
         e.apellido,
         l.nombre,
         TRUNC(v.salida)
ORDER BY e.cedula, fecha_viaje;
```

Resultado:

Results | Explain | Describe | Saved SQL | History

ID_CONDUCTOR	CONDUCTOR	LOCALIDAD	FECHA_VIAJE	TOTAL_VIAJES
1000000017	Estudiante17 Prueba17	Santa Fe	9/4/2026	1
1000000018	Estudiante18 Prueba18	Usaquen	9/1/2026	1
1000000018	Estudiante18 Prueba18	Usaquen	9/2/2026	1
1000000018	Estudiante18 Prueba18	Usaquen	9/3/2026	1
1000000018	Estudiante18 Prueba18	Usaquen	9/4/2026	1
1000000019	Estudiante19 Prueba19	Barrios Unidos	9/1/2026	1
1000000019	Estudiante19 Prueba19	Barrios Unidos	9/2/2026	1
1000000019	Estudiante19 Prueba19	Barrios Unidos	9/3/2026	1
1000000019	Estudiante19 Prueba19	Barrios Unidos	9/4/2026	1
1000000020	Estudiante20 Prueba20	Kennedy	9/1/2026	1
1000000020	Estudiante20 Prueba20	Kennedy	9/2/2026	1
1000000020	Estudiante20 Prueba20	Kennedy	9/3/2026	1
1000000020	Estudiante20 Prueba20	Kennedy	9/4/2026	1

80 rows returned in 0.00 seconds Download

Punto 5

Arquitectura de la aplicación

La aplicación se desarrolló en Oracle APEX 26.2. La autenticación utiliza Oracle APEX Accounts y cada cuenta se asocia a un estudiante mediante la tabla WHEELS_USUARIO_ROL. La autorización se implementó con dos esquemas de tipo Exists SQL Query:

- ES_CONDUCTOR: el usuario autenticado tiene rol CONDUCTOR o AMBOS.
- ES_PASAJERO: el usuario autenticado tiene rol PASAJERO o AMBOS.

Los esquemas se aplican tanto a las páginas como a las entradas del menú de navegación, de modo que cada usuario solo ve y accede a las funcionalidades de su rol. Si un usuario

intenta abrir una página de otro rol, APEX muestra el mensaje del esquema correspondiente.

Página	Nombre	Acceso	Requerimiento
1	Home	Autenticado	–
2	Viajes Disponibles	Pública	RF9
3	Registro de Pasajero	Pública	RF7
4	Mi Perfil	ES_PASAJERO	RF8
5	Solicitar Cupo	ES_PASAJERO	RF10
6	Historial de viajes	ES_CONDUCTOR	RF6
7	Mis Reservas	ES_PASAJERO	RF11
8	Demanda por Localidad	Autenticado	RFC1
9	Ingresos por Localidad	Autenticado	RFC2
10	Gestionar Solicitudes	ES_CONDUCTOR	RF4
11	Viajes por Conductor	Autenticado	RFC3
12	Dashboard Conductor	Autenticado	RFA1
13	Registrar Vehículo	ES_CONDUCTOR	RF2
14	Dashboard Pasajero	Autenticado	RFA2
15	Registro de Conductor	Autenticado	RF1
16	Publicar Viaje	Autenticado	RF3
17	Actualizar Estado del Viaje	Autenticado	RF5
9999	Login	Pública	–

Esquemas de usuario conductor y pasajero

Se verificó que el menú de navegación cambia según el rol del usuario autenticado. El pasajero (estudiante1) no visualiza Historial de viajes, Gestionar Solicitudes ni Registrar Vehículo; el conductor (conductor1) accede a Gestionar Solicitudes y a las demás páginas de su rol.

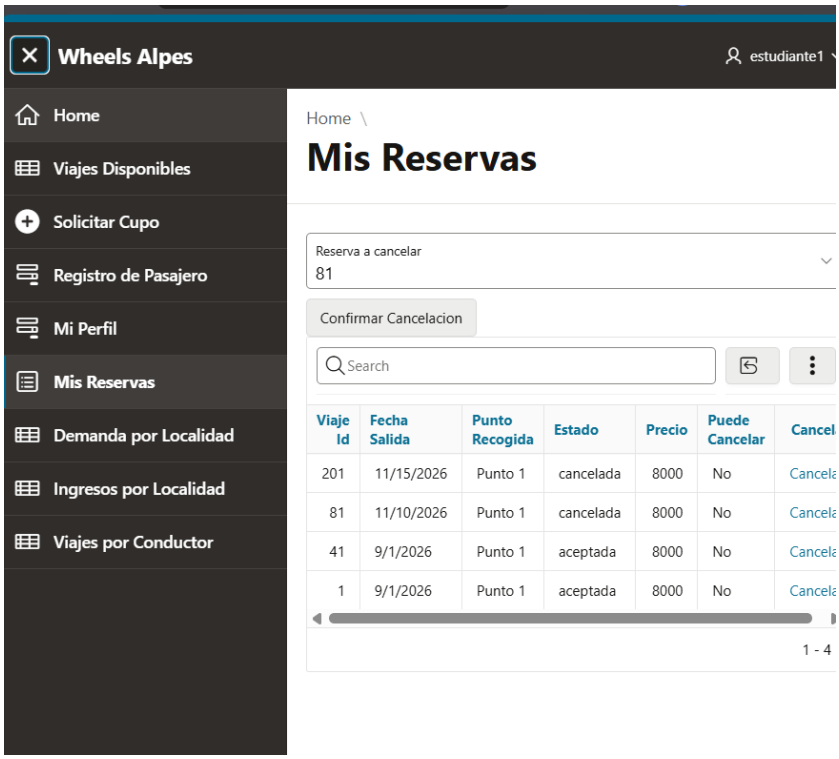


Figura. Menú de navegación del pasajero estudiante1: sin opciones de conductor.

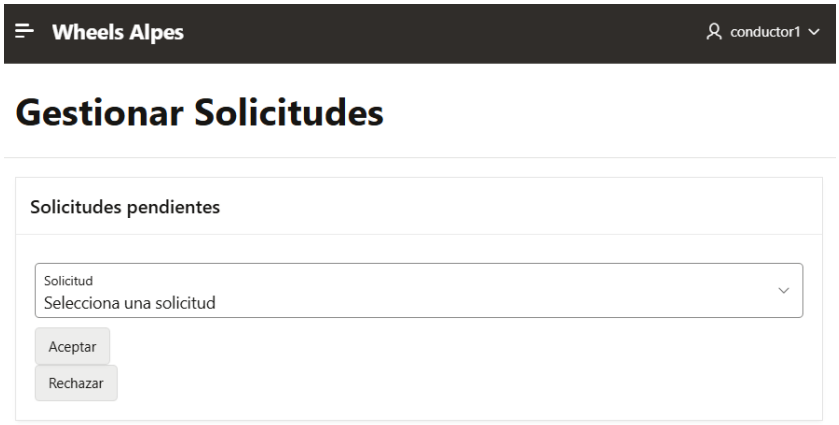


Figura. El conductor conductor1 accede a Gestionar Solicitudes.

Registrar vehículo (RF2)

Objetivo: Permitir que un conductor registre un vehículo con placa, marca, modelo, año, color y número de cupos.
Implementación: Se creó la página 13 mediante un formulario sobre la tabla VEHICULOS. El campo ESTUDIANTE se asigna automáticamente con la cédula del conductor autenticado, consultada en WHEELS_USUARIO_ROL, lo que impide registrar vehículos a nombre de otro estudiante. La página está restringida con el esquema ES_CONDUCTOR.
Resultado: El conductor1 registró el vehículo ABC123 (Mazda 3, gris, 2023, 4 cupos). La aplicación mostró "Row created." y la consulta sobre VEHICULOS confirmó que el vehículo quedó asociado a la cédula 1000000001.

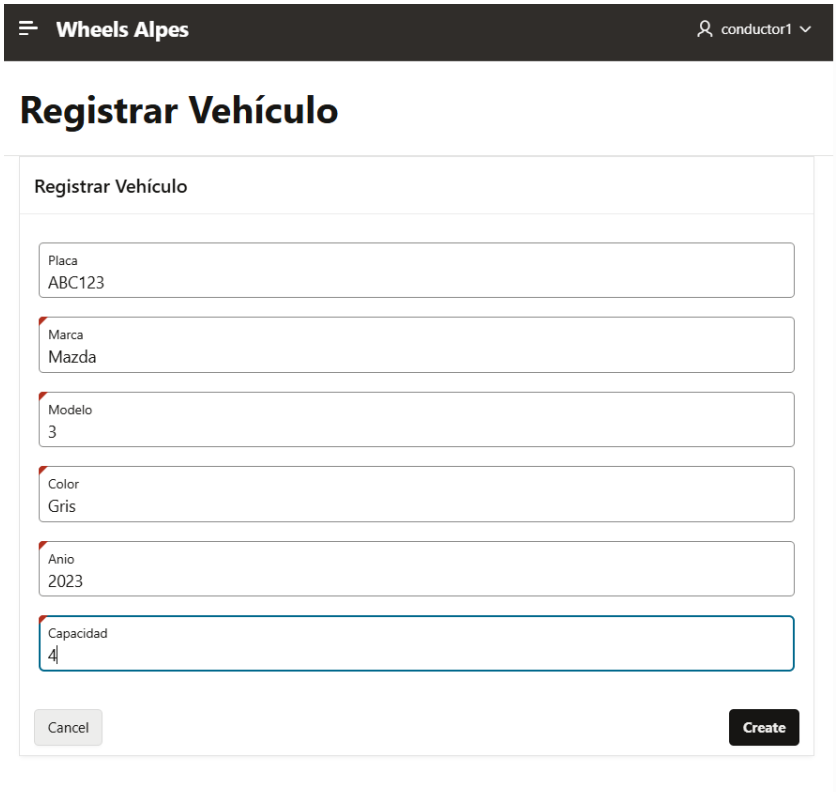


Figura. Formulario de registro de vehículo diligenciado por conductor1.

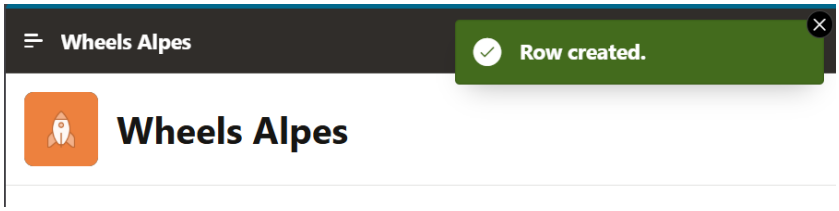


Figura. Mensaje de registro exitoso.



Figura. Verificación en VEHICULOS: el vehículo quedó asociado al conductor 1000000001.

Gestionar solicitudes (RF4)

Objetivo: Permitir al conductor aceptar o rechazar las solicitudes de cupo de sus viajes publicados.
Implementación: Se creó la página 10, Gestionar Solicitudes, con un reporte de las solicitudes pendientes de los viajes programados del conductor autenticado, una lista para seleccionar la solicitud y los botones Aceptar y Rechazar. Un proceso PL/SQL valida que el viaje pertenezca al conductor autenticado, que siga en estado programado, que al aceptar queden cupos disponibles y que la solicitud siga pendiente. El viaje se bloquea con FOR UPDATE para evitar aceptar más pasajeros que los cupos ofrecidos ante operaciones concurrentes.
Resultado: El conductor1 visualizó la solicitud pendiente de Estudiante21 para el viaje 81 y la aceptó. La aplicación mostró "La solicitud fue procesada correctamente" y la consulta sobre RESERVAS confirmó el cambio de estado a aceptada.

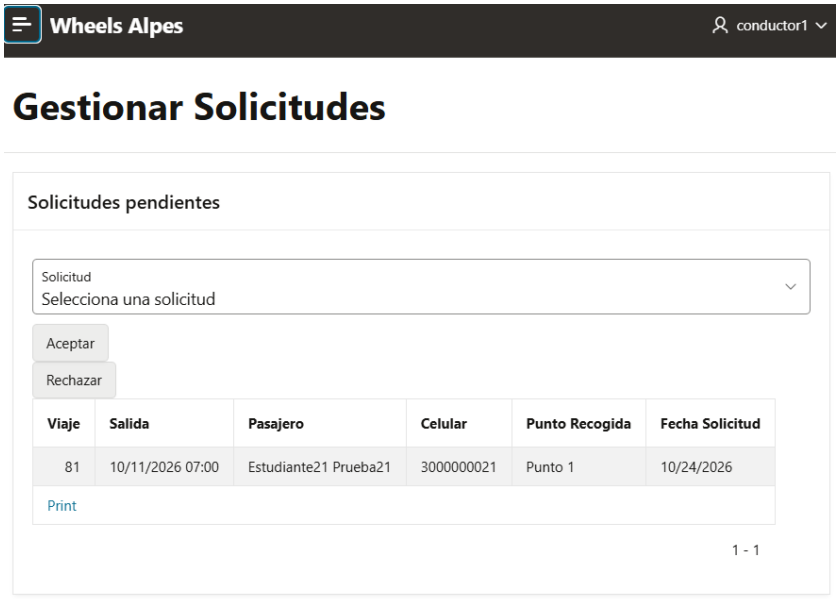


Figura. Solicitudes pendientes del conductor1: Estudiante21 en el viaje 81.

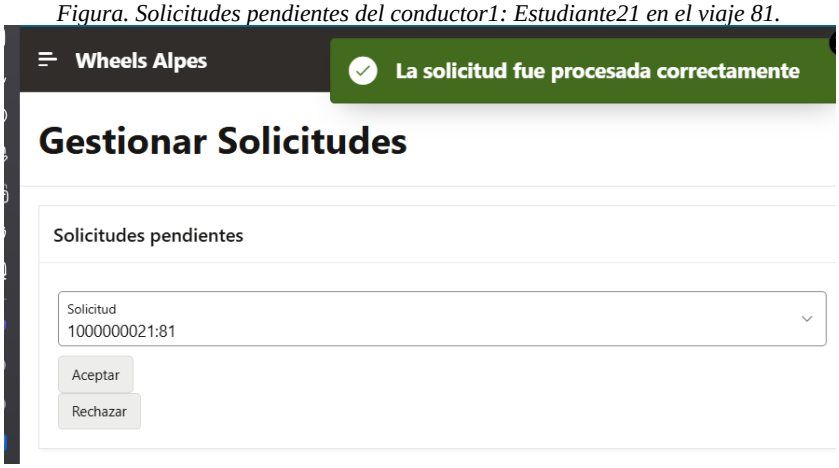


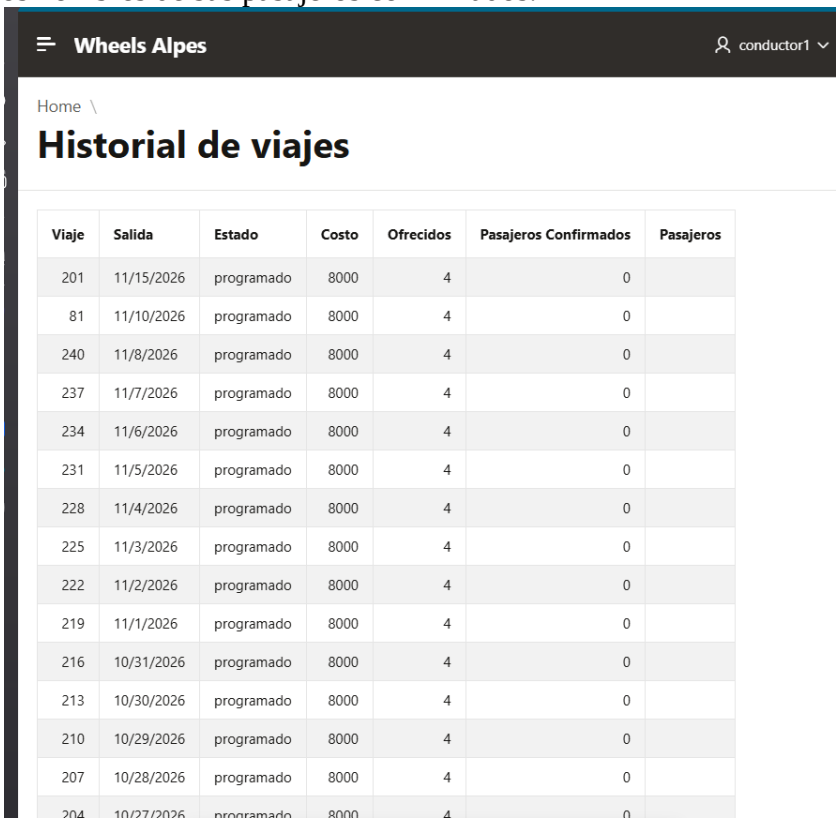
Figura. Solicitud aceptada correctamente.



Figura. Verificación en RESERVAS: la solicitud de 1000000021 en el viaje 81 quedó aceptada.

Consultar historial de viajes (RF6)

Objetivo: Consultar los viajes publicados por el conductor, pasados y futuros, con su estado y sus pasajeros confirmados.
Implementación: Se creó la página 6 mediante un Classic Report restringido con el esquema ES_CONDUCTOR. La consulta relaciona VIAJES, VEHICULOS y WHEELS_USUARIO_ROL para mostrar únicamente los viajes del conductor autenticado. Se utilizó LEFT JOIN con RESERVAS y ESTUDIANTES para conservar los viajes sin reservas; COUNT con CASE cuenta los pasajeros aceptados y LISTAGG muestra sus nombres.
Resultado: Al ingresar como conductor1 se visualizaron únicamente sus viajes: los programados (201, 81 y siguientes) y los completados del 1 al 4 de septiembre de 2026, con el número y los nombres de sus pasajeros confirmados.



Viaje	Salida	Estado	Costo	Ofrecidos	Pasajeros Confirmados	Pasajeros
201	11/15/2026	programado	8000	4	0	
81	11/10/2026	programado	8000	4	0	
240	11/8/2026	programado	8000	4	0	
237	11/7/2026	programado	8000	4	0	
234	11/6/2026	programado	8000	4	0	
231	11/5/2026	programado	8000	4	0	
228	11/4/2026	programado	8000	4	0	
225	11/3/2026	programado	8000	4	0	
222	11/2/2026	programado	8000	4	0	
219	11/1/2026	programado	8000	4	0	
216	10/31/2026	programado	8000	4	0	
213	10/30/2026	programado	8000	4	0	
210	10/29/2026	programado	8000	4	0	
207	10/28/2026	programado	8000	4	0	
204	10/27/2026	programado	8000	4	0	

Figura. Historial del conductor1: viajes programados.



Figura. Historial del conductor1: viajes completados con sus pasajeros confirmados.

Registro de pasajero (RF7)

Objetivo: Permitir el registro de estudiantes como pasajeros dentro de la aplicación Wheels Alpes.
Implementación: Se implementó la página 3, un formulario de Oracle APEX sobre la tabla ESTUDIANTES con nombre, apellido, documento de identidad, correo, celular y programa. El programa se elige de una lista que muestra el programa y su universidad, de modo que también queda registrada la universidad. La página es pública, ya que el registro se realiza antes de tener una cuenta, y no permite eliminar estudiantes.
Resultado: El formulario inserta el estudiante en ESTUDIANTES y las restricciones de la base de datos impiden registrar un documento o un correo ya existentes (PK y UNIQUE). La cuenta de acceso de APEX asociada al estudiante es creada por el administrador del workspace y vinculada en WHEELS_USUARIO_ROL.

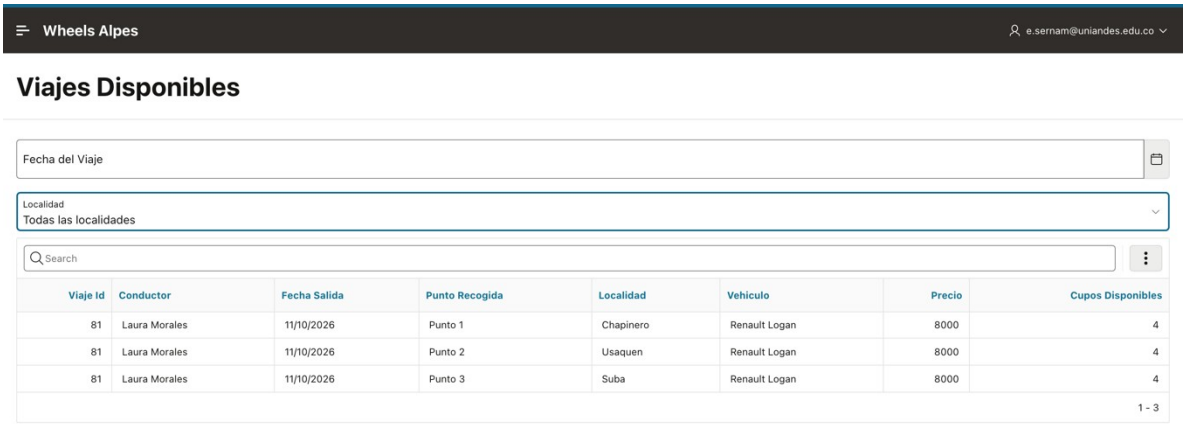
Gestión de perfil (RF8)

Objetivo: Permitir que los pasajeros consulten y actualicen su información personal.
Implementación: Se incorporó la página 4, Mi Perfil, restringida con el esquema ES_PASAJERO. Un proceso PL/SQL previo a la carga identifica la cédula del estudiante asociado a la sesión mediante WHEELS_USUARIO_ROL y carga sus datos en el formulario. Otro proceso PL/SQL actualiza nombre, apellido, correo, celular y programa únicamente para ese estudiante, y genera un error controlado si no se actualizó ninguna fila. El documento de identidad no se modifica, porque es la clave primaria de ESTUDIANTES y está referenciado por otras tablas.

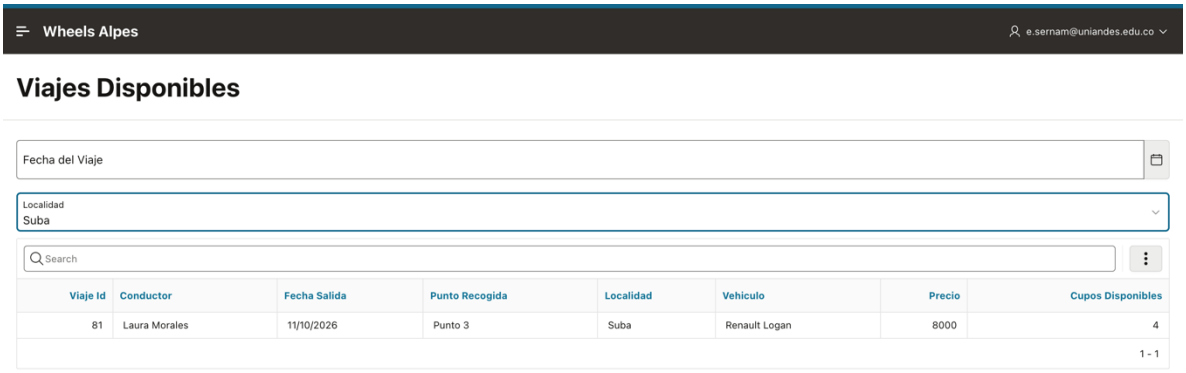
Consultar viajes disponibles (RF9)

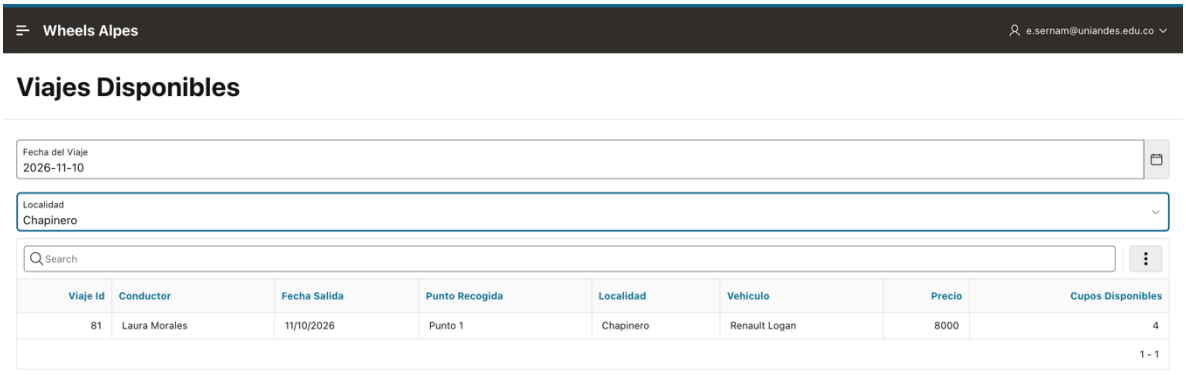
Objetivo: Permitir la búsqueda de viajes disponibles a partir de una fecha y una localidad de recogida, sin necesidad de autenticación.

Implementación: Se desarrolló un Interactive Report en la página 2, que es pública. La consulta relaciona VIAJES, VEHICULOS, ESTUDIANTES, RUTA_PUNTOS, PUNTOS, LOCALIDADES y RESERVAS. Se utilizaron INNER JOIN para recuperar la información del conductor, el vehículo y los posibles puntos de recogida. Se implementaron filtros por fecha y localidad, además de condiciones sobre el estado del viaje, la fecha de salida y los cupos disponibles.
Resultado: Sin filtro, el reporte muestra el viaje 81 de Laura Morales con sus tres posibles puntos de recogida. Al filtrar por la localidad Suba se obtiene únicamente el punto ubicado en esa localidad, y al filtrar por la fecha 2026-11-10 y la localidad Chapinero se obtiene el viaje 81 con tarifa de $8.000 y cuatro cupos disponibles.
Sin filtro:



Con filtro:





Solicitud de cupo (RF10)

Objetivo: Permitir que un pasajero autenticado solicite un cupo en un viaje disponible, seleccionando un punto de recogida.
Implementación: Se implementó la página 5, Solicitar Cupo, restringida con el esquema ES_PASAJERO, con un formulario y un proceso PL/SQL. El sistema identifica al pasajero asociado a la cuenta autenticada y permite seleccionar el viaje y uno de los puntos de su ruta. Antes de registrar la solicitud, el proceso valida que:

- el viaje esté programado y su salida sea futura;
- el pasajero no sea el conductor del viaje;
- el punto de recogida pertenezca a la ruta del viaje;
- el pasajero no tenga ya una solicitud para ese viaje;
- queden cupos disponibles.

El viaje se bloquea con FOR UPDATE durante la validación, y la solicitud se registra en RESERVAS con estado pendiente.
Resultado: Se realizó una solicitud para el viaje 201, con recogida en el Punto 1 de Chapinero. La aplicación mostró un mensaje de registro exitoso y la reserva quedó almacenada en RESERVAS con estado pendiente. Posteriormente, se intentó realizar una segunda solicitud sobre el mismo viaje y la aplicación rechazó la operación con el error controlado ORA-20005, indicando que el pasajero ya tenía una solicitud registrada.

Cancelar solicitud (RF11)

Objetivo: Permitir que el pasajero cancele una reserva confirmada con al menos dos horas de anticipación.
Implementación: La página 7, Mis Reservas, restringida con el esquema ES_PASAJERO, muestra las reservas del pasajero autenticado e indica si cada una puede cancelarse. Una lista desplegable presenta únicamente las reservas aceptadas de viajes programados cuya salida está a más de dos horas. Al confirmar, un proceso PL/SQL vuelve a validar la identidad del pasajero y las condiciones de cancelación, bloqueando el viaje y la reserva, antes de actualizar el estado a cancelada.
Resultado: Se probó el flujo completo con datos reales de la aplicación: la solicitud de estudiante1 para el viaje 81 fue aceptada por conductor1 mediante RF4; en Mis Reservas la reserva apareció como aceptada y cancelable; al confirmar la cancelación, la aplicación mostró "Tu reserva fue cancelada correctamente" y el estado cambió a cancelada.

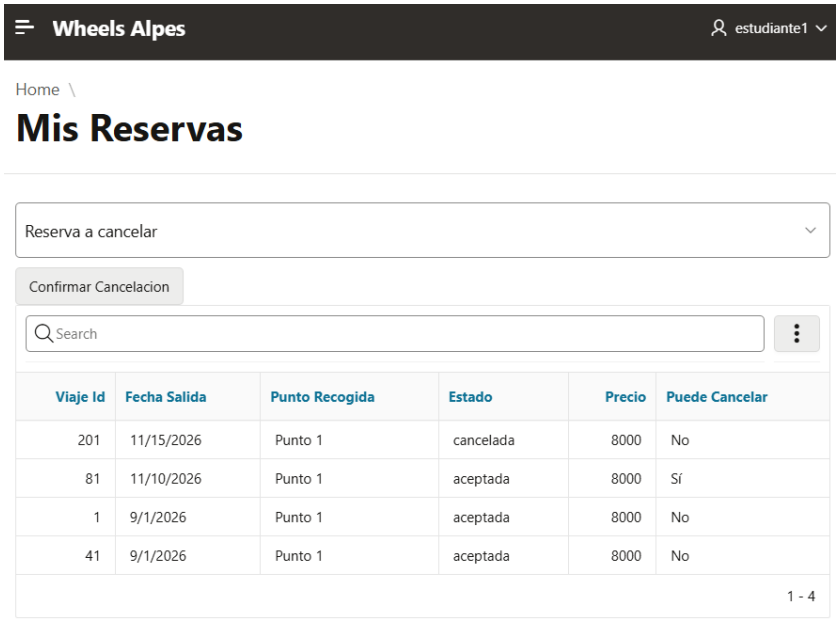


Figura. Mis Reservas: el viaje 81 aparece aceptado y cancelable.

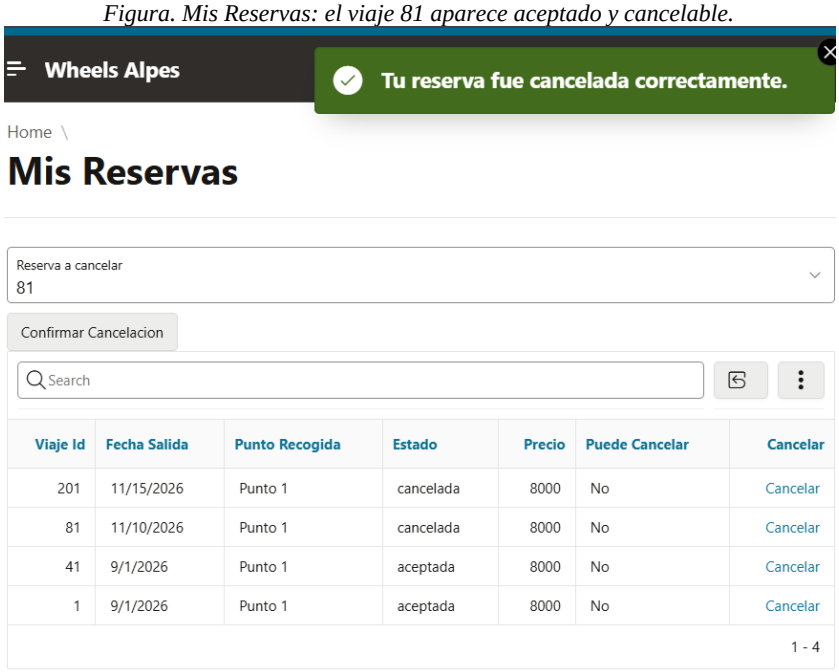


Figura. Reserva del viaje 81 cancelada correctamente.

Requerimientos adicionales: RF1, RF3, RF5, RFA1 y RFA2

En la etapa final del desarrollo se incorporaron a la aplicación las siguientes páginas, que completan los requerimientos funcionales del conductor y los tableros de control:
RF1 – Registro de Conductor: formulario para registrar la información personal del conductor (nombre, apellidos, programa y universidad, correo institucional, celular y documento de identidad).

RF3 – Publicar Viaje: formulario para crear un viaje indicando fecha y hora de salida, punto de origen, ruta, cupos ofrecidos, tarifa y vehículo asociado. El viaje se crea en estado programado.
RF5 – Actualizar Estado del Viaje: permite al conductor avanzar el estado del viaje siguiendo la secuencia programado → en curso → completado, o cancelarlo, sin modificar viajes ya completados o cancelados.
RFA1 – Dashboard Conductor: tablero con los indicadores del conductor sobre sus viajes completados: total de pasajeros transportados, tiempo total, ingresos y distancia recorrida.
RFA2 – Dashboard Pasajero: tablero con los indicadores del pasajero: número de viajes tomados, zona en la que más ha solicitado viajes, total de gastos y promedio de distancia y tiempo por viaje.

Punto 6 – Escenarios de prueba

Pruebas de integridad

Prueba	Caso exitoso	Caso de falla	Resultado obtenido
Unicidad de PK	Registro del vehículo ABC123	Registrar nuevamente la placa ABC123	ORA-00001: unique constraint PK_VEHICULOS violated
Integridad referencial (FK)	Vehículo ABC123 asociado al estudiante existente 1000000001	INSERT de un vehículo con el estudiante inexistente 9999999999	ORA-02291: FK_VEHICULOS_ESTUDIANTE violated – parent key not found
Restricción de chequeo	Vehículo ABC123 con capacidad 4	Vehículo DEF456 con capacidad 0	ORA-02290: CK_VEHICULOS_CAPACIDAD violated

En los tres casos el primer registro se insertó correctamente (ver RF2) y el SMBD rechazó la operación inválida, que era el resultado esperado.

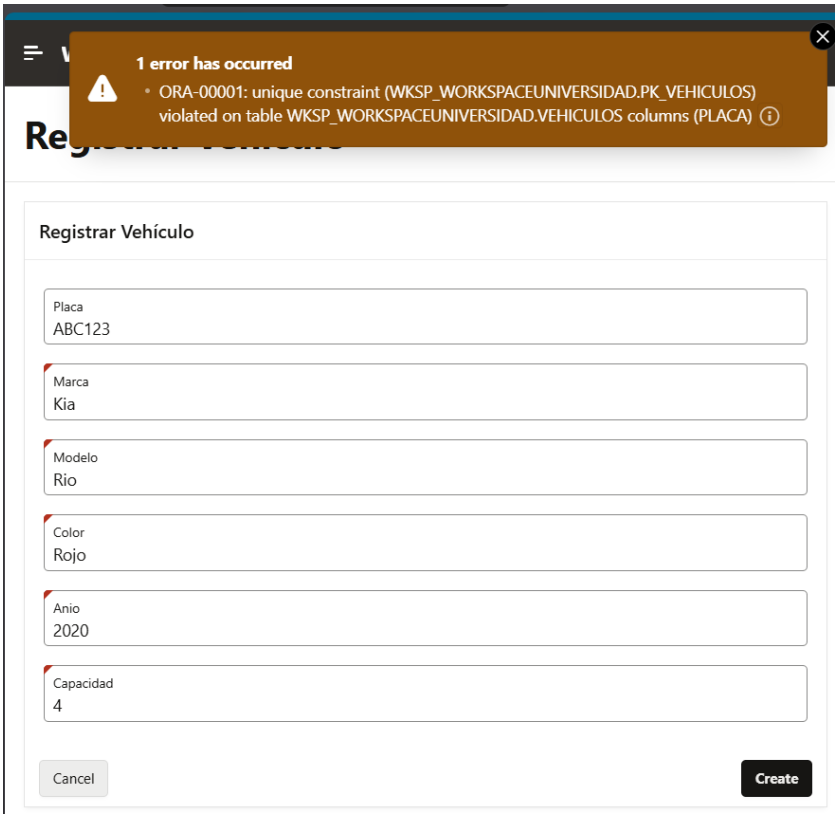


Figura. ORA-00001 al registrar una placa existente desde la aplicación.

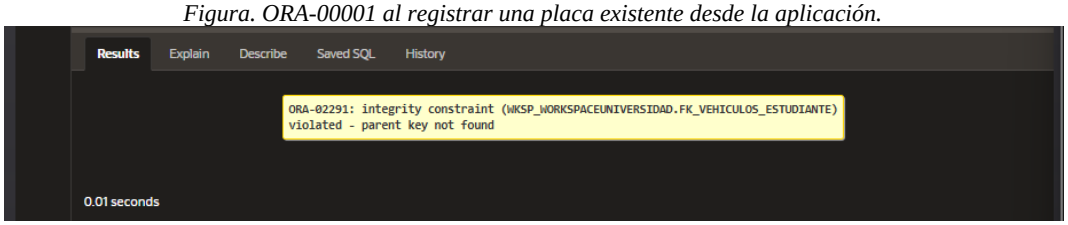


Figura. ORA-02291 al insertar un vehículo con un estudiante inexistente.

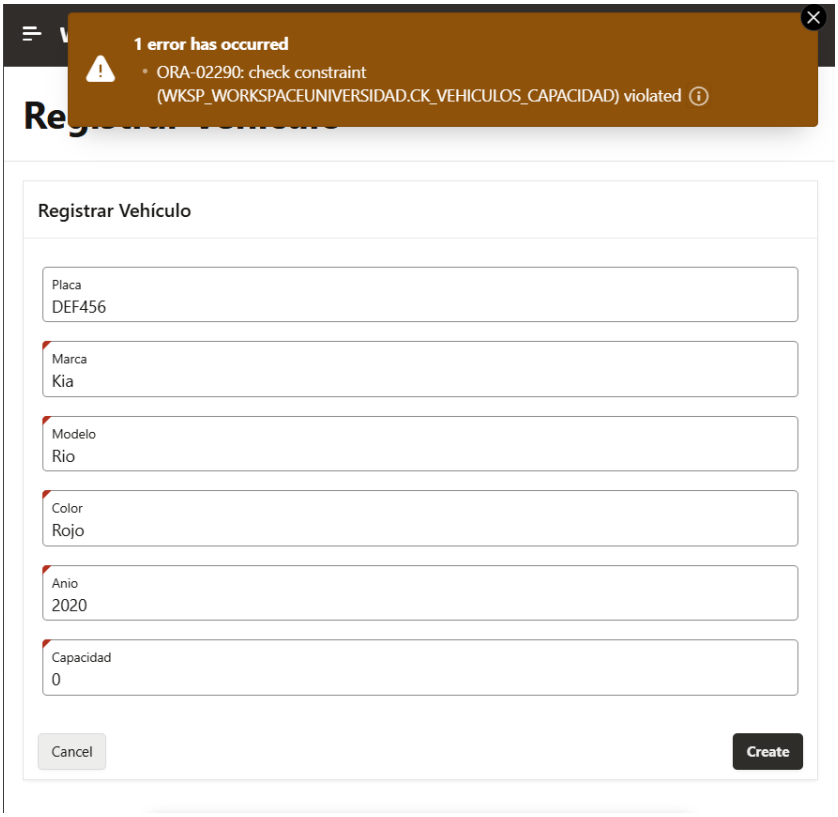


Figura. ORA-02290 al registrar un vehículo con capacidad 0 desde la aplicación.

Pruebas funcionales y de reglas de negocio

Escenario	Usuario	Resultado esperado	Resultado obtenido
Solicitar cupo en un viaje disponible (RF10)	estudiante1	Reserva registrada como pendiente	Correcto
Solicitar cupo dos veces en el mismo viaje (RF10)	estudiante1	Rechazo con ORA-20005	Correcto
Aceptar una solicitud pendiente (RF4)	conductor1	La reserva pasa a aceptada	Correcto
Cancelar una reserva aceptada con más de 2 h de anticipación (RF11)	estudiante1	La reserva pasa a cancelada	Correcto
Registrar vehículo asignado al conductor autenticado (RF2)	conductor1	ESTUDIANTE = 1000000001	Correcto
Consultar historial propio (RF6)	conductor1	Solo los viajes del conductor	Correcto
Menú según rol (esquemas de usuario)	estudiante1	Sin opciones de conductor	Correcto

Balance del plan de pruebas

Logrado: Se implementaron en la aplicación los requerimientos funcionales RF1 a RF11, los esquemas de usuario conductor y pasajero, los requerimientos de consulta RFC1, RFC2 y RFC3, y los tableros de control RFA1 y RFA2. Se verificaron las restricciones de unicidad, integridad referencial y chequeo, y el flujo completo solicitud → aceptación → cancelación con datos reales de la aplicación.

Observaciones: Respecto a RF5, se identificó una inconsistencia en el enunciado: la cancelación desde el estado "en curso" con dos horas de anticipación no es posible, porque un viaje en curso ya inició; por ello, la cancelación se permite para viajes programados con

al menos dos horas de anticipación. El registro de pasajero (RF7) crea el estudiante, pero la cuenta de acceso de APEX debe ser creada por el administrador del workspace. Las capturas de prueba documentadas se concentran en RF2, RF4, RF6, RF9, RF10 y RF11, y en las pruebas de integridad.